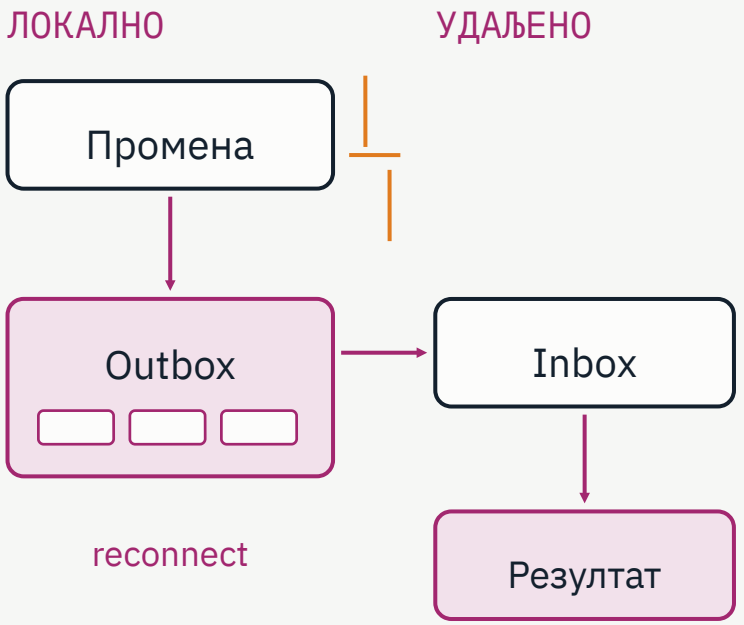
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Рад без везе и поуздана испорука

Очување намере кроз прекид и reconnect



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–22

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

23–30

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Разумети како buffer, outbox, дедупликација и replay раде заједно.

Прекид везе раздваја тренутак настанка поруке од тренутка пријема. Систем мора да изабере шта чува, шта понавља и како спречава дуплу обраду.

НАПОМЕНА

Поуздана испорука даје поруци видљив животни циклус, уместо да се ослања на претпоставку да мрежа неће направити грешку.

01 · ПОЈМОВИ

Четири појма теме

Прекид везе не мора да заустави локални рад, али уводи питање шта се десило док друга страна није била доступна.

01 **Локална намера**

настаје и чува се

02 **Outbox или buffer**

чека сигурну испоруку

03 **Прималац**

памти обрађене поруке

04 **DLQ и replay**

видљива корекција неуспеха

01 · ПОЈМОВИ

Локална намера

Када нема везе, систем најпре трајно чува намеру и њен идентитет. Тако корисникова радња не нестаје само зато што удаљени систем тренутно није доступан.

01 Трајно се чува пре покушаја испоруке.

02 Радња не нестаје због недоступности удаљеног система.

03 Има сопствени стабилан идентитет.

01 · ПОЈМОВИ

Outbox или buffer

Outbox или buffer чека нову прилику за испоруку уз ограничен капацитет и јасну политику када се пуни. Бесконачан ред само прикрива проблем.

01 Ограничен капацитет, не бесконачан ред.

02 Има јасну политику за случај попуњености.

03 Чека нову прилику за испоруку, не губи поруку.

01 · ПОЈМОВИ

Дедупликација

Прималац памти стабилан идентитет већ обрађене поруке. Тиме reconnect и поновна испорука не стварају други пословни ефекат.

01 Памти идентитет већ обрађене поруке.

02 Спречава други пословни ефекат при поновној испоруци.

03 Идентитет мора бити стабилан, не променљив податак.

01 · ПОЈМОВИ

Replay

Replay понавља видљиво неуспешну испоруку кроз исти ток валидације и аудита. Не сме бити пречица која заобилази правила зато што је порука стара.

01 Понавља испоруку кроз исти ток валидације.

02 Не заобилази правила зато што је порука стара.

03 Пролази и кроз audit као свака друга порука.

Пример: reconnect после локалног рада

Удаљена компонента наставља локални рад док нема везу и чува поруке за касније.

01 Buffer има ограничење и јасно правило када више нема простора.

02 Outbox чува намеру заједно са пословном променом коју представља.

03 Прималац води евиденцију обрађених порука, а dead-letter чува неуспехе за контролисан replay.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема рада без везе

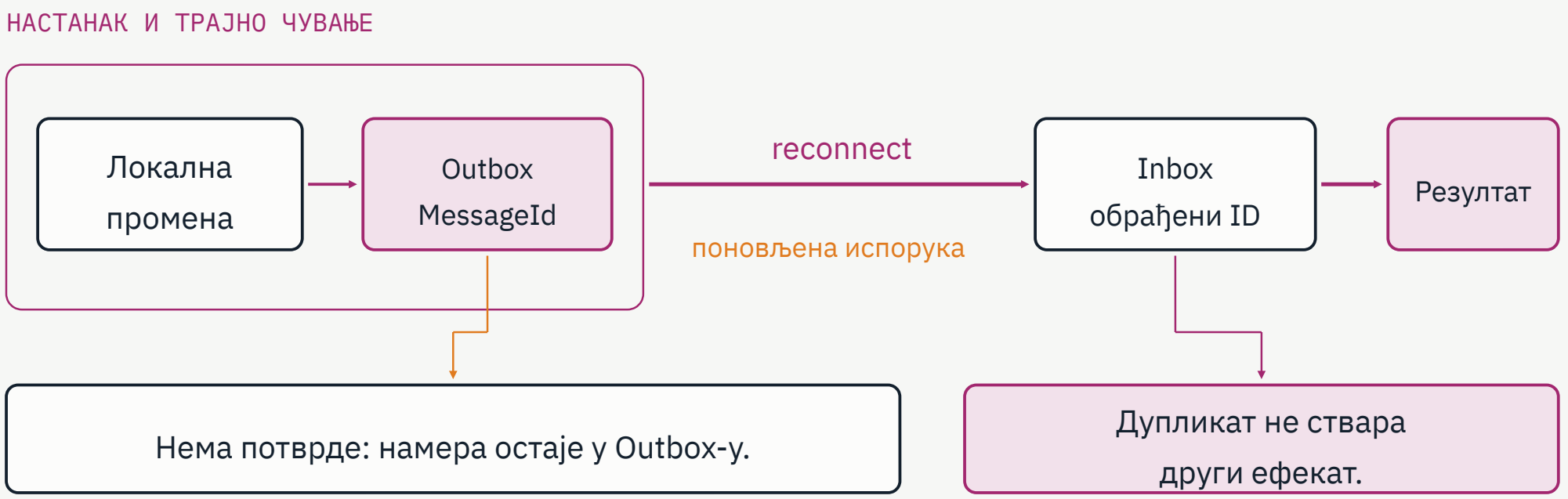
Прекид везе мења тренутак испоруке, али не сме да избрише намеру нити да је произвољно удвостручи.

Елемент	Улога у току
Локална промена	настаје без мреже
Buffer или outbox	чува поруку и разлог
Reconnect	понавља испоруку
Inbox и резултат	спречавају дупли ефекат

02 · ДИЈАГРАМ

Намера преживљава прекид везе

Локална промена и Outbox чувају намеру, а Inbox спречава дупли ефекат после reconnect-а.



ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример чува намеру кроз прекид везе.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/07 Rad bez veze outbox"
dotnet run --project src/Odp.Vezba07.ConsoleUi
dotnet test Odp.Vezba07.sln
```

Контекст сценарија

Правило у коду

Упис мерења успева и када везе нема, слање је одвојен корак.

ПРИКАЗАНИ ПРИМЕР

Flush шаље поруке редом и означава их испорученим тек када потврда стигне.

Правило у коду

Flush шаље поруке редом и означава их испорученим тек када потврда стигне.

C# · доменско правило

```
1   public FlushReport Flush()
2   {
3       var pending = ordering.PendingInOrder(outbox.All());
4       if (pending.Count == 0)
5           return new(0, 0, OutboxCodes.NothingPending);
6       var delivered = 0;
7       foreach (var message in pending)
8       {
9           if (!uplink.TrySend(message))
10              return new(delivered, pending.Count - delivered, OutboxCodes.DeliveryUnconfirmed);
```

Правило у коду (наставак)

Flush шаље поруке редом и означава их испорученим тек када потврда стигне.

C# · доменско правило

```
            outbox.MarkDelivered(message.Position);
            delivered++;
        }
        return new(delivered, 0, OutboxCodes.Flushed);
    }
```

Контекст сценарија

Тест који задаје квар

После повратка везе поруке стижу редом којим су настале.

ПРИКАЗАНИ ПРИМЕР

Изгубљена потврда изазива retry, а центар поруке примењује једном и задржава њихов редослед.

Тест који задаје квар

Изгубљена потврда изазива retry, а центар поруке примењује једном и задржава њихов редослед.

C# · NUnit

```
1   [Test]
2   public void Flush_WhenAckIsLost_ResendsAndTheCenterAppliesTheMessageOnce()
3   {
4       _record.Record("m-1", "a");
5       _record.Record("m-2", "b");
6       _uplink.LoseNextAck = true;
7       var first = _flush.Flush();
8       var second = _flush.Flush();
9       using (Assert.EnterMultipleScope())
10      {
11          Assert.That(first, Is.EqualTo(new FlushReport(0, 2, OutboxCodes.DeliveryUnconfirmed)));
```

Тест који задаје квар (наставак)

Изгубљена потврда изазива retry, а центар поруке примењује једном и задржава њихов редослед.

C# · NUnit

```
12          Assert.That(second, Is.EqualTo(new FlushReport(2, 0, OutboxCodes.Flushed)));
13          Assert.That(_center.AppliedPayloads, Is.EqualTo(new[] { "a", "b" }));
14          Assert.That(_center.DuplicatesIgnored, Is.EqualTo(1));
15      }
16  }
```

Испис демонстрације

Изгубљена потврда изазива поновно слање, а центар поруку примењује једном.

console · исходи демонстрације

```
Veza je u prekidu.
  merenje m-1 -> message_queued
  merenje m-2 -> message_queued
  merenje m-3 -> message_queued
  pokušaj slanja bez veze -> delivery_unconfirmed, isporučeno 0, preostalo 3
Veza se vratila, ali se prva potvrda gubi.
  prvi pokušaj posle povratka -> delivery_unconfirmed, isporučeno 0, preostalo 3
  drugi pokušaj -> outbox_flushed, isporučeno 3, preostalo 0
Centar je primenio 3 poruke redom:
  signal -91 dBm
  signal -88 dBm
  signal -97 dBm
Ignorisanih duplikata: 1
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Порука која не пролази не нестаје тихо, већ добија видљив разлог.
- Поновљена испорука је очекивана и прималац је препознаје.
- Локални капацитет и политика одбацивања морају бити експлицитни.

ЗАМКЕ

- Buffer без ограничења и без политике при попуњености.
- Replay који заобилази исту валидацију као први покушај.
- Дедупликација ослоњена само на податак који се може променити.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Buffer без ограничења.

02 Replay који заобилази валидацију.

03 Дедупликација заснована на променљивом податку.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Одредити капацитет и политику при попуњености.

02 Поновити повезивање након прекида.

03 Пратити поруку до коначног исхода.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Коју поруку бисте обавезно сачували док нема везе, а коју бисте смели да одбаците уз обавештење?

ЗА ВЕЖБУ

Коју поруку бисте морали сачувати током прекида везе, а коју бисте смели изгубити уз јасно обавештење? Зашто?

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Ограничити величину outbox-а (Capacity).

02 Када је пун, ново мерење се одбија кодом outbox_full уместо да се тихо изгуби старо.

03 Правило покрити тестом у коме веза остаје у прекиду.

04 · ПРИМЕНА

Примена на пројекат

Принцип са вежбе примењује се на додељену пројектну целину.

01 Идентификовати конкретну дистрибуирану одлуку у својој целини.

02 Имплементирати је са стабилним кодом исхода.

03 Припремити поновљив failure сценарио и тест који га доказује.

Резиме

Тим најпре одлучује шта може да се задржи локално, колико дуго и шта се дешава при попуњеном капацитету.

Прекид везе мења тренутак испоруке, али не сме да избрише намеру нити да је произвољно удвостручи.

НАПОМЕНА

Наредна вежба: Координација, проток и евентуална конзистентност.